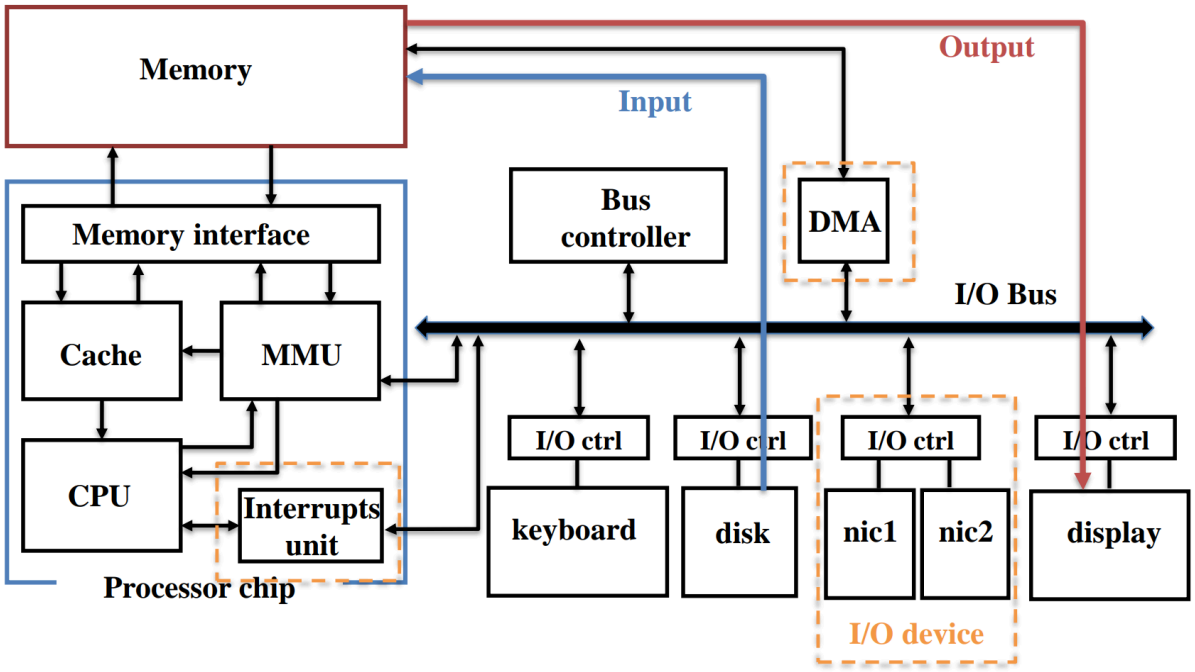
I/O Devices

Description

The I/O operations have a significant impact on the total program execution time.

The example below shows the effect of speeding up the CPU while leaving the I/O operations unchanged

$$Speedup = \frac{Exec.timebeforeenhancement}{Exec.timeafterenhancement}$$



Amdahl's law

It describes the potential maximum speedup achievable by a system (or program) when a portion of the it is optimized

> 💡 *In parallel computing the Amdahl's law is one of the reference upper-bound of the parallelization process*

$$Speedup = \frac{Exec.time before enhancement}{Exec.time after enhancement} = \frac{T1 - f + Tf}{T1 - f + \frac{Tf}{N}} = \frac{1}{(1 - f) + \frac{f}{N}}$$

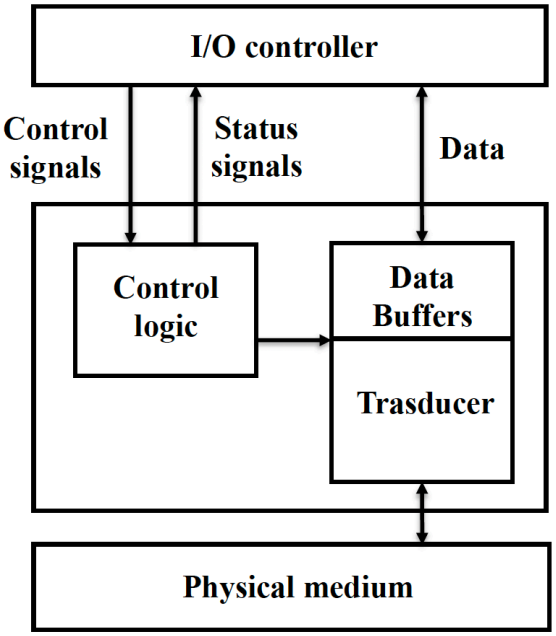
Performance of the I/O subsystem is very important, but it is not all. Other important aspects are:

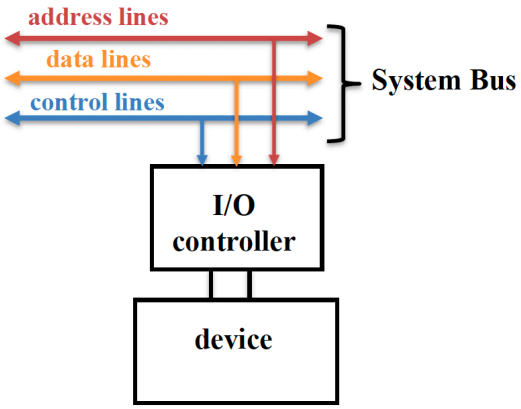
- **Reliability**
  - Mean Time To/Between Failure (MTTF/MTBF) metrics
  - Fault avoidance (better components)
  - Fault tolerance (introducing some level of redundancy)
    - Example: RAID (Redundant Array of Independent Disks) in critical storage systems
- **Availability**
  - Mean Time To Repair (*MTTR*)

$$Availability = \frac{MTTF}{MTTF + MTTR}$$

I/O ports

- **Control**: used for sending commands and read status report
- **Data**: Used to transfer bulk data to/from the device memory

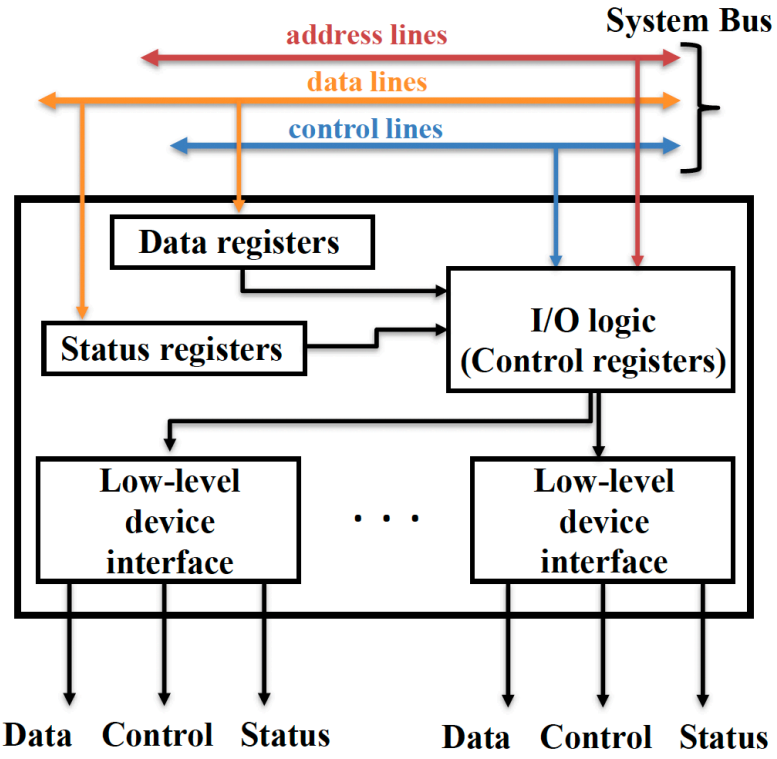




An I/O controller is a component that manages communication between the CPU, memory, and peripheral devices, ensuring efficient data transfer

It acts as intermediaries between the bus and peripheral devices

- ***Registers*** store and manage data, commands, and status information
- ***Low-level*** interfaces handle electrical signals to perform I/O operations



I/O Functions

- **Control and timing**
  Manages the flow of data and signals between the processor, memory, and I/O device
  Ensures synchronization between components to avoid data loss or conflicts

- **Processor communication**
  - Command decoding (e.g., read sector X)
  - Data exchange: facilitates data transfer between the processor and I/O device.
  - Status reporting (e.g., ready, busy, error)
  - Address recognition: Identifies the specific device or memory location being addressed

- **Device communication**: exchanges commands, status info, data

- **Data buffering**
  To optimize data transfers and compensate different speeds, temporarily stores data to accommodate the speed mismatch between devices and the system bus

- **Error detection**
  - Transmission errors (parity checks)
  - Electrical/mechanical errors

Bus interconnection

A collection of data lines treated together as a single logical signalUsed to describe a shared collection of lines connecting multiple devices (connected devices are called ***taps***)



Performance factors: physical length, number of connected taps

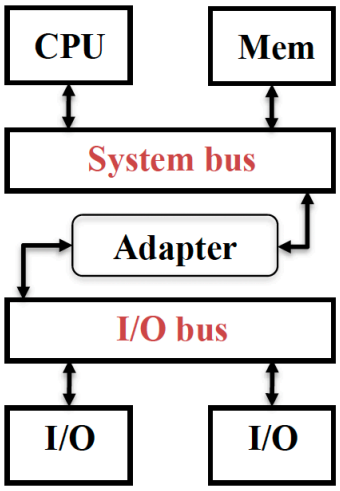
A bus is a ***shared transmission medium, o*****nly one device at a time can successfully transmit**

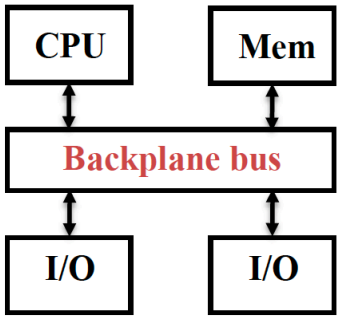
Bus **arbitration** is required to manage access when multiple devices want to transmit simultaneously.



The **system bus** connects major components (processor, memory, I/O bus)

Examples:





- **System bus:** Connects processor and memory; I/O interfaced through adapters
- **I/O bus:** Connects I/O devices; no direct connection with processor and memory
- **Backplane bus:** Connects CPU, memory, I/O devices

Bus Clocking

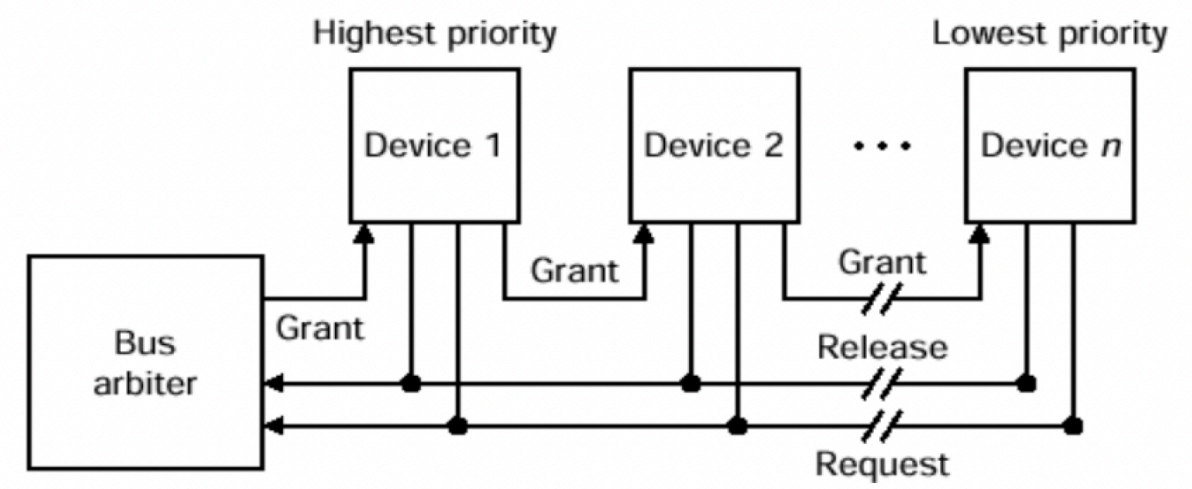
- **Synchronous**: All devices connected to the bus share the same bus clock. Events happen at the edge of the clock signal
- **Asynchronous**: The bus has no clock, so no clock skew. It relies on **handshaking protocols** to synchronize communication between devices.

Bus arbitration

- **Bus master**: *unit that can initiate a bus request*
- **Arbitration**: choosing a master among multiple requests trying to implement priority and **fairness** (preventing starvation)

- *Centralized*: a dedicated device (Arbiter) collects requests and then decide (potential *bottleneck problem*)
- *Distributed*: every device sees the requests simultaneously and participates in the selection of the next master (more complex to realize and it needs a lot of control lines)

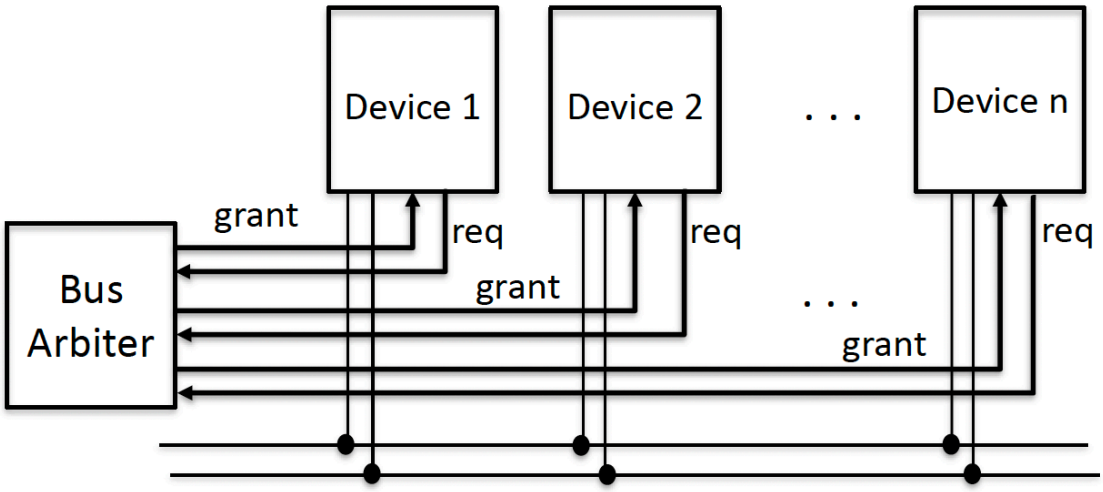
Daisy-chain arbiter



Devices connect to the bus in priority order, with the highest priority device closest to the arbiter.

Simple to implement but slow and cannot ensure fairness

Centralized arbiter



Centralized arbiter with independent request/grant channels for each device

The arbiter determines which devices get access based on

- **Priority**: some devices may have higher priority (priority may be fixed or dynamic)
- **Round-robin**: devices are served in a cyclic order to ensure fairness

Bus Examples

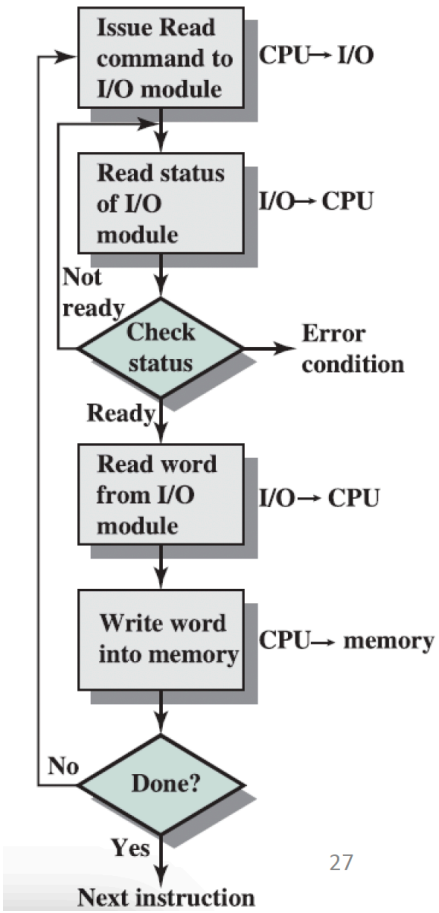
Characteristic	Firewire (1394)	USB 2.0	PCI Express	Serial ATA	Serial Attached SCSI
Intended use	External	External	Internal	Internal	External
Devices per channel	63	127	1	1	4
Basic data width (signals)	4	2	2 per lane	4	4
Theoretical peak bandwidth	50 MB/sec (Firewire 400) or 100 MB/sec (Firewire 800)	0.2 MB/sec (low speed), 1.5 MB/sec (full speed), or 60 MB/sec (high speed)	250 MB/sec per lane (1x); PCIe cards come as 1x, 2x, 4x, 8x, 16x, or 32x	300 MB/sec	300 MB/sec
Hot pluggable	Yes	Yes	Depends on form factor	Yes	Yes
Maximum bus length (copper wire)	4.5 meters	5 meters	0.5 meters	1 meter	8 meters
Standard name	IEEE 1394, 1394b	USB Implementors Forum	PCI-SIG	SATA-IO	T10 committee

Querying I/O Status

Programmed I/O: The CPU directly controls I/O operation continuously monitors the device status

Interrupt-driven I/O: The CPU is notified **asynchronously** by an interrupt signal from an I/O device when it is ready or has completed an operation

> 💡 An ***interrupt*** is an *asynchronous event coming from a device*



Polling

CPU waits for I/O module to complete operations through a ***polling*** mechanism

Polling means the CPU constantly checks the device, even when no data is ready

```
// READ example:
write I/O control register for READ op
while (ready-bit == 0) do ; // busy waiting
load the data in memory
```

Pros:
Simple to implement as it requires minimal HW support

Cons:
Inefficient use of CPU time (waste of processor's cycles)

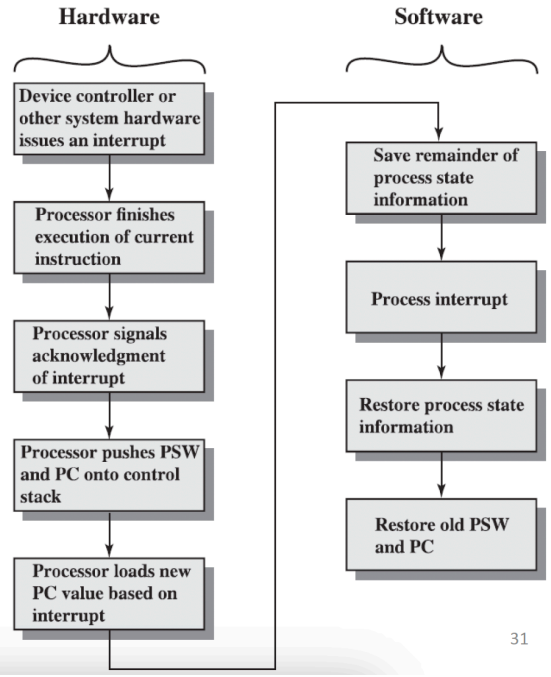
Interrupts

An alternative to pollingThe CPU issues commands to a device and continues performing other (useful) tasks.

When the I/O device changes status (data is ready), it generates an interrupt to notify the CPU

Interrupt management step

```
while(true) {
  try {
    IR = fetch(PC);
    decode(IR);
    execute(IR);
    update(PC);
  } catch (exception e) {
    exception_management();
  }
  if(interrupt) {
    interrupt_management();
  }
}
```



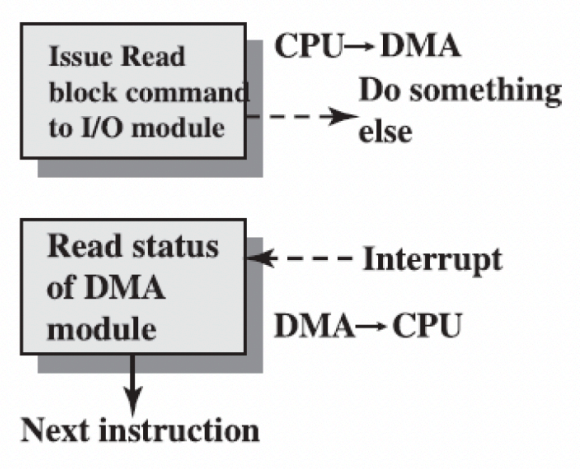


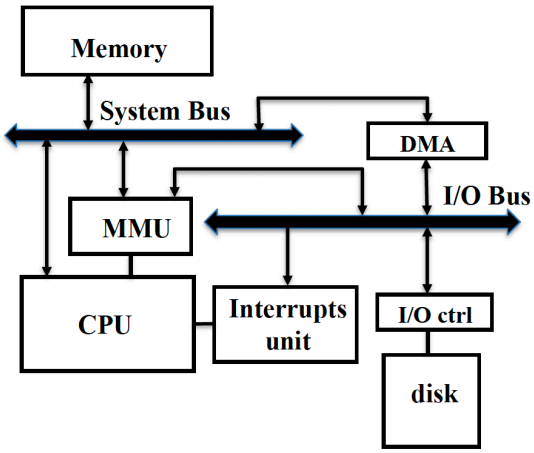
PSW is the Program Status Word register, it holds info about processor's state

Direct Memory Access

A mechanism that allows I/O device controllers to transfer data **directly to and from main memory *without involving the CPU***, separates the CPU-Memory communication from the I/O device-memory protocol.

Is implemented with a specialized controller attached to the system bus


Issue Read block command to I/O module
CPU→DMA
Do something else
Read status of DMA module
Interrupt
DMA→CPU
Next instruction


Memory
System Bus
DMA
I/O Bus
MMU
CPU
Interrupts unit
I/O ctrl
disk